Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №7
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Авдей Д.Б.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить возможности языка программирования Python в разработке оконных приложений.

Вариант №1

Задание 1. Построение графических примитивов и надписей

Создать класс Triangle и класс Point. Объявить список из n объектов класса Point, написать функцию, определяющую, какая из точек лежит внутри, а какая – снаружи треугольника.

Код:

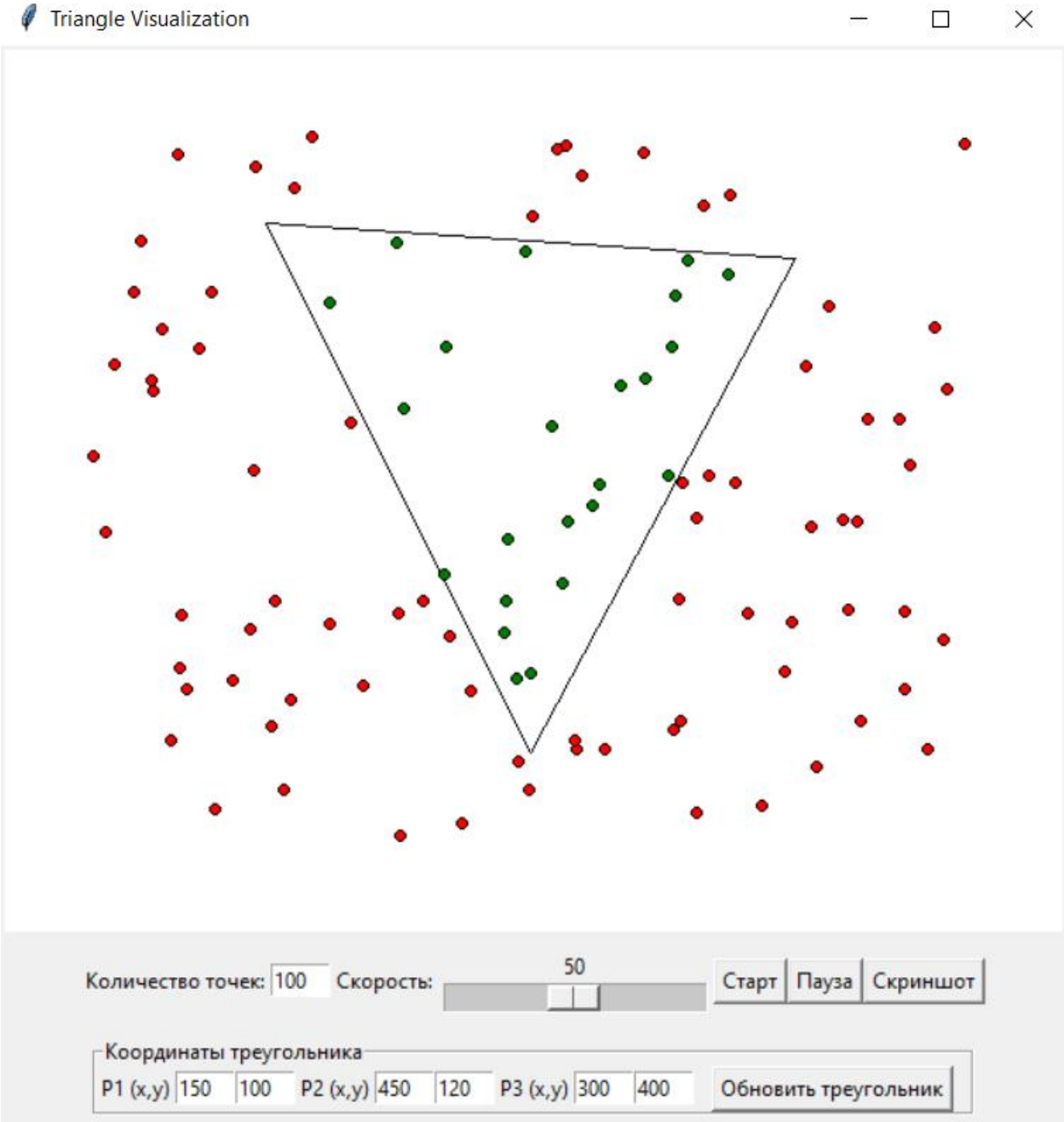
```
import tkinter as tk
import random
import time
from PIL import ImageGrab


class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y


class Triangle:
    def __init__(self, p1, p2, p3):
        self.p1 = p1
        self.p2 = p2
        self.p3 = p3

    def area(self, a, b, c):
        return abs((a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y -
b.y)) / 2.0)

    def contains(self, point):
        A = self.area(self.p1, self.p2, self.p3)
        A1 = self.area(point, self.p2, self.p3)
        A2 = self.area(self.p1, point, self.p3)
        A3 = self.area(self.p1, self.p2, point)

        return abs(A - (A1 + A2 + A3)) < 0.5


class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Triangle Visualization")

        self.canvas = tk.Canvas(root, width=600, height=500, bg="white")
        self.canvas.pack()
```

```
        controls = tk.Frame(root)
        controls.pack(pady=5)

        # Количество точек
        tk.Label(controls, text="Количество точек:").grid(row=0, column=0)
        self.points_entry = tk.Entry(controls, width=5)
        self.points_entry.insert(0, "20")
        self.points_entry.grid(row=0, column=1)

        # Скорость
        tk.Label(controls, text="Скорость:").grid(row=0, column=2)
        self.speed = tk.Scale(controls, from_=1, to=100,
orient=tk.HORIZONTAL, length=150)
        self.speed.set(50)
        self.speed.grid(row=0, column=3)

        tk.Button(controls, text="Старт", command=self.start).grid(row=0,
column=4)
        tk.Button(controls, text="Пауза", command=self.pause).grid(row=0,
column=5)
        tk.Button(controls, text="Скриншот",
command=self.screenshot).grid(row=0, column=6)

        # Ввод координат треугольника
        triangle_frame = tk.LabelFrame(root, text="Координаты треугольника")
        triangle_frame.pack(pady=5)

        tk.Label(triangle_frame, text="P1 (x,y)").grid(row=0, column=0)
        self.p1x = tk.Entry(triangle_frame, width=5)
        self.p1y = tk.Entry(triangle_frame, width=5)
        self.p1x.insert(0, "150")
        self.p1y.insert(0, "100")
        self.p1x.grid(row=0, column=1)
        self.p1y.grid(row=0, column=2)

        tk.Label(triangle_frame, text="P2 (x,y)").grid(row=0, column=3)
        self.p2x = tk.Entry(triangle_frame, width=5)
        self.p2y = tk.Entry(triangle_frame, width=5)
        self.p2x.insert(0, "450")
        self.p2y.insert(0, "120")
        self.p2x.grid(row=0, column=4)
        self.p2y.grid(row=0, column=5)

        tk.Label(triangle_frame, text="P3 (x,y)").grid(row=0, column=6)
        self.p3x = tk.Entry(triangle_frame, width=5)
        self.p3y = tk.Entry(triangle_frame, width=5)
        self.p3x.insert(0, "300")
        self.p3y.insert(0, "400")
        self.p3x.grid(row=0, column=7)
        self.p3y.grid(row=0, column=8)

        tk.Button(triangle_frame, text="Обновить треугольник",
command=self.update_triangle).grid(
            row=0, column=9, padx=10
        )

        self.running = False
        self.update_triangle()

    def update_triangle(self):
        try:
            self.triangle = Triangle(
                Point(int(self.p1x.get()), int(self.p1y.get())),
```

```
                Point(int(self.p2x.get()), int(self.p2y.get())),
                Point(int(self.p3x.get()), int(self.p3y.get())),
            )
            self.canvas.delete("all")
            self.draw_triangle()
        except ValueError:
            pass

    def draw_triangle(self):
        self.canvas.create_polygon(
            self.triangle.p1.x,
            self.triangle.p1.y,
            self.triangle.p2.x,
            self.triangle.p2.y,
            self.triangle.p3.x,
            self.triangle.p3.y,
            outline="black",
            fill="",
        )

    def start(self):
        self.running = True
        self.canvas.delete("all")
        self.draw_triangle()

        n = int(self.points_entry.get())
        self.points = [Point(random.randint(50, 550), random.randint(50,
450)) for _ in range(n)]

        self.animate()

    def animate(self):
        if not self.running:
            return

        self.canvas.delete("points")
        self.draw_triangle()

        for point in self.points:
            color = "green" if self.triangle.contains(point) else "red"

            self.canvas.create_oval(point.x - 3, point.y - 3, point.x + 3,
point.y + 3, fill=color, tags="points")

        delay = 101 - self.speed.get()
        self.root.after(delay * 10, self.animate)

    def pause(self):
        self.running = not self.running
        if self.running:
            self.animate()

    def screenshot(self):
        self.canvas.update()

        x = self.canvas.winfo_rootx()
        y = self.canvas.winfo_rooty()
        x1 = x + self.canvas.winfo_width()
        y1 = y + self.canvas.winfo_height()

        ImageGrab.grab().crop((x, y, x1,
y1)).save(f"screenshot_{int(time.time())}.png")
```

```
root = tk.Tk()
app = App(root)
root.mainloop()
```

Результат:



Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

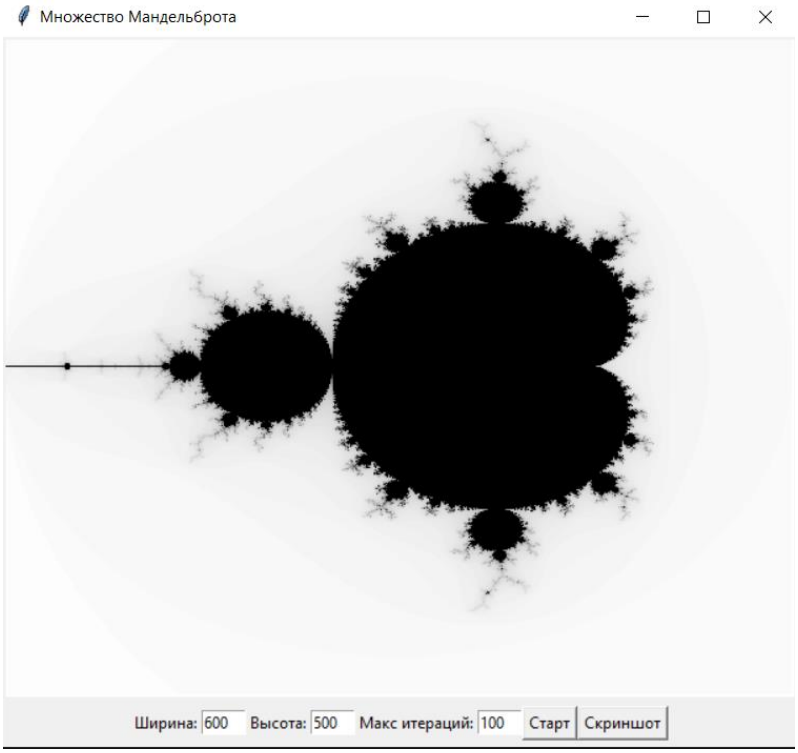
1) Множество Мальдеброта

Код:

```
import tkinter as tk
from PIL import Image, ImageTk, ImageGrab
import time


class MandelbrotApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Множество Мандельброта")

        self.canvas = tk.Canvas(root, width=600, height=500, bg="white")
        self.canvas.pack()

        # Панель управления
        controls = tk.Frame(root)
        controls.pack(pady=5)

        tk.Label(controls, text="Ширина:").grid(row=0, column=0)
        self.width_entry = tk.Entry(controls, width=5)
        self.width_entry.insert(0, "600")
        self.width_entry.grid(row=0, column=1)

        tk.Label(controls, text="Высота:").grid(row=0, column=2)
        self.height_entry = tk.Entry(controls, width=5)
        self.height_entry.insert(0, "500")
        self.height_entry.grid(row=0, column=3)

        tk.Label(controls, text="Макс итераций:").grid(row=0, column=4)
        self.iter_entry = tk.Entry(controls, width=5)
        self.iter_entry.insert(0, "100")
        self.iter_entry.grid(row=0, column=5)

        tk.Button(controls, text="Старт", command=self.start).grid(row=0,
column=6)
        tk.Button(controls, text="Скриншот",
command=self.screenshot).grid(row=0, column=7)

        self.image = None

    def mandelbrot(self, c, max_iter):
        z = 0
        for n in range(max_iter):
            if abs(z) > 2:
                return n
            z = z * z + c
        return max_iter

    def generate(self, width, height, max_iter):
        self.img = Image.new("RGB", (width, height))
        pixels = self.img.load()

        re_start, re_end = -2.0, 1.0
        im_start, im_end = -1.5, 1.5

        for x in range(width):
            for y in range(height):
                c = complex(re_start + (x / width) * (re_end - re_start),
im_start + (y / height) * (im_end - im_start))
                m = self.mandelbrot(c, max_iter)
                color = 255 - int(m * 255 / max_iter)
```

```
                pixels[x, y] = (color, color, color)

            if x % 20 == 0:
                self.update_canvas()

        self.update_canvas()

    def update_canvas(self):
        self.photo = ImageTk.PhotoImage(self.img)
        self.canvas.create_image(0, 0, image=self.photo, anchor=tk.NW)
        self.root.update()

    def start(self):
        width = int(self.width_entry.get())
        height = int(self.height_entry.get())
        max_iter = int(self.iter_entry.get())

        self.canvas.config(width=width, height=height)
        self.generate(width, height, max_iter)

    def screenshot(self):
        self.canvas.update()

        x = self.canvas.winfo_rootx()
        y = self.canvas.winfo_rooty()
        x1 = x + self.canvas.winfo_width()
        y1 = y + self.canvas.winfo_height()

        ImageGrab.grab().crop((x, y, x1,
y1)).save(f"screenshot_mandelbrot_{int(time.time())}.png")


root = tk.Tk()
app = MandelbrotApp(root)
root.mainloop()
```

Результат:



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.